

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA THÀNH PHỐ HỒ CHÍ MINH
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



DSA-Extended

Báo cáo bài tập lớn (Mở rộng)
PII-NER-ANONYMIZER

GVHD: Vương Bá Thịnh

Sinh viên thực hiện:	Trần Đình Ý	2414100
	Trần Văn Minh Triết	2413603
	Nguyễn Phúc Vĩnh	2414001
	Nguyễn Đình Minh	2412079

HỒ CHÍ MINH, THÁNG 5/ 2026

Tóm tắt nội dung

Trong thời đại số hiện nay, lượng dữ liệu văn bản chứa thông tin cá nhân ngày càng gia tăng và được sử dụng rộng rãi trong nhiều hệ thống khác nhau như mạng xã hội, thương mại điện tử, giáo dục và y tế. Điều này làm cho việc bảo vệ dữ liệu cá nhân trở thành một yêu cầu quan trọng trong các ứng dụng xử lý ngôn ngữ tự nhiên. Một trong những hướng tiếp cận phổ biến là sử dụng Named Entity Recognition (NER) để phát hiện và ẩn danh các thông tin định danh cá nhân trong văn bản.

Project này tập trung xây dựng một hệ thống nhận diện và ẩn danh thông tin cá nhân bằng phương pháp rule-based, không sử dụng machine learning hoặc deep learning. Hệ thống sử dụng các kỹ thuật xử lý chuỗi như regular expression, keyword matching, token scanning và context-based detection để phát hiện các thực thể quan trọng gồm NAME, EMAIL, PHONE, ADDRESS, USERNAME và URL. Sau khi phát hiện, các thông tin nhạy cảm sẽ được thay thế bằng các placeholder tương ứng nhằm giảm nguy cơ lộ dữ liệu cá nhân nhưng vẫn giữ được cấu trúc tổng quát của văn bản.

Pipeline của hệ thống bao gồm các bước preprocessing dữ liệu, chuyển đổi BIO labels thành ground truth entities, phát hiện entity bằng regex, phát hiện entity bằng ngữ cảnh, xử lý overlap giữa các entity, thực hiện anonymization và đánh giá kết quả bằng Precision, Recall và F1-score. Dataset được sử dụng trong project là [PII External Dataset](#) từ Kaggle, cung cấp dữ liệu văn bản cùng nhãn BIO phục vụ cho việc xây dựng ground truth và đánh giá kết quả của hệ thống.

Hệ thống cho thấy khả năng phát hiện tương đối tốt đối với các thực thể có cấu trúc cố định như email, URL và số điện thoại. Đồng thời, project cũng thể hiện vai trò của các thuật toán xử lý chuỗi trong bài toán NER và bảo vệ dữ liệu cá nhân. Mặc dù vẫn còn một số hạn chế về khả năng tổng quát hóa và xử lý các ngữ cảnh phức tạp, hệ thống đã xây dựng được một pipeline tương đối hoàn chỉnh cho bài toán phát hiện và ẩn danh thông tin cá nhân trong văn bản.

Từ khóa: Named Entity Recognition, PII Detection, Rule-based Method, Regular Expression, String Matching, Text Anonymization, Natural Language Processing.

Mục lục

Tóm tắt nội dung	i
Mục lục	ii
Danh sách hình vẽ	iii
Danh sách bảng	iv
Danh sách đoạn mã	v
1 Giới thiệu	1
2 Phương pháp Rule-based cho bài toán NER	3
2.1	3
2.2 Nhận diện tên riêng và địa chỉ (Context Detector)	4
2.2.1 Nhận diện tên người	4
2.2.2 Nhận diện địa chỉ	9
2.3 Hợp nhất thực thể (Merger)	10
2.3.1 Vấn đề overlap	11
2.3.2 Hệ thống ưu tiên	11
2.3.3 Thuật toán Greedy Priority-First	11
2.3.4 Minh họa trường hợp overlap	12
3 Tổng kết	14
4 Hướng mở rộng trong tương lai	15

Danh sách hình vẽ

Danh sách bảng

2.1	Bảng tín hiệu chấm điểm cho candidate tên người	6
2.2	Danh sách thực thể trước khi hợp nhất	12
2.3	Danh sách thực thể sau khi hợp nhất	13

Danh sách đoạn mã

**PHÂN CÔNG NHIỆM VỤ VÀ KẾT QUẢ CỦA TỪNG
THÀNH VIÊN**

STT	HỌ VÀ TÊN	MSSV	NHIỆM VỤ	KẾT QUẢ
1	Trần Văn Minh Triết	2413603	None.	100%
2	Nguyễn Đình Minh	2412079	None.	100%
3	Nguyễn Phúc Vĩnh	2414001	None.	100%
4	Trần Đình Ý	2414100	None.	100%

1 Giới thiệu

Named Entity Recognition, được viết tắt là NER, là một bài toán quan trọng trong lĩnh vực xử lý ngôn ngữ tự nhiên. Mục tiêu chính của NER là phát hiện và phân loại các thực thể có ý nghĩa trong văn bản, chẳng hạn như tên người, địa điểm, tổ chức, email, số điện thoại, địa chỉ, tên tài khoản hoặc đường dẫn URL. Thay vì chỉ xem văn bản như một chuỗi ký tự thông thường, NER giúp hệ thống hiểu được những thành phần nào trong văn bản mang thông tin quan trọng và chúng thuộc loại nào.

Trong thực tế, NER được ứng dụng trong nhiều bài toán khác nhau. Ví dụ, trong hệ thống tìm kiếm thông tin, NER giúp trích xuất tên người, địa danh hoặc tổ chức để cải thiện độ chính xác khi truy vấn. Trong lĩnh vực chăm sóc khách hàng, NER có thể dùng để nhận diện thông tin đơn hàng, địa chỉ giao hàng hoặc số điện thoại của người dùng. Trong các hệ thống quản lý tài liệu, NER hỗ trợ tự động phân loại và trích xuất thông tin quan trọng từ văn bản. Đặc biệt, đối với bài toán bảo vệ dữ liệu cá nhân, NER có vai trò phát hiện các thông tin định danh cá nhân, từ đó phục vụ cho việc ẩn danh hoặc che giấu dữ liệu nhạy cảm trước khi lưu trữ, chia sẻ hoặc xử lý tiếp.

Trong phạm vi project này, bài toán được tập trung vào việc nhận diện và ẩn danh thông tin cá nhân trong văn bản. Các loại thông tin cần phát hiện bao gồm **NAME**, **EMAIL**, **PHONE**, **ADDRESS**, **USERNAME** và **URL**. Đây là những dạng dữ liệu có khả năng định danh trực tiếp hoặc gián tiếp một cá nhân. Ví dụ, một đoạn văn bản như:

`My name is John Smith and my email is john@gmail.com.`

sau khi xử lý có thể được chuyển thành:

`My name is [NAME] and my email is [EMAIL].`

Cách xử lý này giúp giảm nguy cơ lộ thông tin cá nhân, đồng thời vẫn giữ lại cấu trúc tổng quát của văn bản để phục vụ cho các bước phân tích tiếp theo.

Có nhiều hướng tiếp cận khác nhau để giải quyết bài toán NER. Hướng thứ nhất là sử dụng các phương pháp học máy hoặc học sâu, trong đó mô hình được huấn luyện trên dữ liệu đã gán nhãn để tự học cách nhận diện thực thể. Các mô hình hiện đại có thể đạt độ chính xác cao, đặc biệt khi có lượng dữ liệu lớn và tài nguyên tính toán phù hợp. Tuy nhiên, hướng tiếp cận này thường yêu cầu quá trình huấn luyện phức tạp, phụ thuộc nhiều vào dữ liệu, và khó giải thích rõ từng quyết định của hệ thống.

Hướng thứ hai là sử dụng phương pháp rule-based, tức là xây dựng các luật thủ công dựa trên đặc điểm chuỗi, biểu thức chính quy và ngữ cảnh xuất hiện của thực thể. Ví

dụ, email thường có ký tự @ và tên miền phía sau, URL thường bắt đầu bằng `http://`, `https://` hoặc `www.`, còn số điện thoại thường là chuỗi chữ số có độ dài nhất định và có thể chứa dấu cách, dấu gạch ngang hoặc mã quốc gia. Đối với các thực thể khó hơn như tên người hoặc địa chỉ, hệ thống có thể dựa vào các từ khóa ngữ cảnh như `my name is`, `full name`, `address`, `street`, `road`, `city`, v.v.

Trong project này, nhóm lựa chọn hướng rule-based thay vì machine learning hoặc deep learning. Lý do là project tập trung vào việc áp dụng các thuật toán xử lý chuỗi, regular expression, keyword matching, token scanning và xử lý khoảng giao nhau giữa các entity. Cách tiếp cận này phù hợp với mục tiêu học thuật vì dễ hiện thực, dễ kiểm soát, dễ giải thích và không cần quá trình huấn luyện mô hình. Ngoài ra, rule-based method cũng cho phép từng thành viên trong nhóm phát triển các module độc lập, sau đó tích hợp lại thành một pipeline hoàn chỉnh.

Pipeline tổng quát của hệ thống gồm các bước chính. Đầu tiên, dữ liệu thô được đọc từ file CSV và xử lý ở bước preprocessing. Trong bước này, các token và nhãn BIO trong dataset được chuyển thành danh sách ground truth entities để phục vụ đánh giá. Tiếp theo, hệ thống sử dụng regex-based detector để nhận diện các thực thể có cấu trúc rõ ràng như email, số điện thoại và URL. Sau đó, context-based detector được dùng để nhận diện các thực thể phụ thuộc nhiều vào ngữ cảnh như tên người, địa chỉ và username. Kết quả từ hai detector được đưa vào module merger để gộp, loại bỏ trùng lặp và xử lý các entity bị chồng lấn. Cuối cùng, hệ thống thực hiện ẩn danh văn bản và đánh giá kết quả bằng các chỉ số Precision, Recall và F1-score.

Tóm lại, bài toán NER trong project này không chỉ có ý nghĩa trong việc nhận diện thông tin từ văn bản, mà còn gắn liền với nhu cầu bảo vệ dữ liệu cá nhân. Việc xây dựng một hệ thống rule-based giúp nhóm hiểu rõ hơn về cách biểu diễn dữ liệu văn bản, cách phát hiện mẫu chuỗi, cách xử lý entity overlap và cách đánh giá kết quả nhận diện. Đây là nền tảng quan trọng để mở rộng sang các hệ thống NER phức tạp hơn trong tương lai.



2 Phương pháp Rule-based cho bài toán NER

2.1

2.2 Nhận diện tên riêng và địa chỉ (Context Detector)

Module `context_detector` chịu trách nhiệm phát hiện ba loại thực thể dựa trên ngữ cảnh xung quanh: tên người (`NAME`), tên người dùng (`USERNAME`) và địa chỉ (`ADDRESS`). Khác với `regex_detector` vốn chỉ dựa vào cấu trúc cố định của chuỗi ký tự, module này phân tích ý nghĩa ngữ nghĩa của văn bản để xác định thực thể. Phần dưới đây tập trung trình bày chi tiết hai thành phần chính: nhận diện tên người và nhận diện địa chỉ.

2.2.1 Nhận diện tên người

Nhận diện tên người là bài toán khó nhất trong ba loại thực thể vì tên người không tuân theo một cấu trúc cố định và có thể xuất hiện trong rất nhiều ngữ cảnh khác nhau. Thuật toán được thiết kế theo hai tầng xử lý: *fast path* khi có dấu hiệu rõ ràng, và *scoring path* khi cần suy luận từ ngữ cảnh.

Tiền xử lý — Tách câu thông minh. Trước khi phân tích ngữ cảnh, văn bản đầu vào được tách thành các câu riêng lẻ. Đây là bước quan trọng vì điểm ngữ cảnh (pronoun, nghề nghiệp, trigger) chỉ có ý nghĩa trong phạm vi một câu.

Thách thức chính là phân biệt dấu chấm kết thúc câu với dấu chấm trong các trường hợp khác. Hàm `_split_sentences(text)` xử lý điều này theo quy tắc: dấu `.` chỉ được coi là ranh giới câu khi *đồng thời* thỏa ba điều kiện:

1. Token kết thúc bằng `.` không thuộc danh sách viết tắt đã biết (`Dr.`, `Mr.`, `St.`, `Apt.`, `Inc.`, v.v.).
2. Phần văn bản ngay sau `.` không bắt đầu bằng tên miền cấp cao (`com`, `edu`, `vn`, `org`, v.v.) — nhằm tránh split nhầm địa chỉ email và URL.
3. Sau dấu `.` là khoảng trắng hoặc hết chuỗi.

Dấu `!` và `?` luôn được coi là kết thúc câu vô điều kiện.

Tầng 1 — Fast path (Trigger mạnh). Nếu văn bản chứa một trong các cụm từ trigger, tên được xác định ngay lập tức mà không cần qua bước chấm điểm. Trigger được chia thành hai nhóm:

- **NAME_TRIGGERS:** cụm từ chỉ thẳng tên chủ thể, gồm `"my name is"`, `"full name"`, `"name:"`.
- **_LABEL_TRIGGERS:** nhãn hay đứng trước tên trong văn bản có cấu trúc, gồm khoảng 25 mẫu như `"contact information:"`, `"author:"`, `"submitted by:"`, `"regards,"`, `"sincerely,"`, v.v.

Sau khi tìm thấy trigger, thuật toán thu thập 1–4 token liên tiếp thỏa: bắt đầu bằng chữ hoa, không chứa chữ số, không chứa ký tự đặc biệt (ngoại trừ dấu gạch ngang – vì tên có thể có dạng **Mary-Jane**), không phải stopword, và không mang đuôi động từ sau token đầu tiên (**-ing**, **-tion**, **-ed**, v.v.).

Tầng 2 — Scoring path (Chấm điểm ngữ cảnh). Khi không có trigger, thuật toán chuyển sang scoring path gồm ba bước.

Bước 2.1 — Quét candidate. Hàm `_extract_name_candidates(text)` duyệt toàn bộ văn bản, tìm các cụm từ viết hoa liên tiếp có độ dài từ 2 đến 4 token. Các điều kiện loại trừ được áp dụng tại hai mức:

- *Outer loop* (loại trước khi thu thập): token bắt đầu bằng chữ số, token trước là số nguyên thuần túy (dấu hiệu địa chỉ), là title prefix (**Dr.**, **Mr.**, v.v. — strip và tiếp tục), là **As** đứng đầu câu, là stopword, hoặc là từ tiêu đề section (**Information**, **Details**, **Witness**, v.v.).
- *Inner loop* (dừng thu thập giữa chừng): gặp token thường (không hoa), chứa ký tự đặc biệt, là chức danh công việc (**Manager**, **Director**, **Account**, v.v.), mang đuôi động từ sau token đầu tiên, hoặc bắt đầu bằng chữ số.

Sau khi thu thập xong, một candidate bị loại nếu token cuối cùng là hậu tố chỉ tổ chức hoặc địa điểm (**University**, **Company**, **Hospital**, **Hotel**, v.v.) — nhóm này gồm khoảng 70 từ chia thành bốn loại: công ty/doanh nghiệp, tổ chức/cơ quan, trường học, và địa điểm/cơ sở vật chất.

Bước 2.2 — Chấm điểm. Hàm `_score_candidate(candidate, sentence, full_text)` tính điểm cho từng cặp (candidate, câu). Một candidate xuất hiện qua nhiều câu sẽ lấy điểm cao nhất trong tất cả các câu đó. Bảng 2.1 tổng hợp các tín hiệu và điểm tương ứng.

Bảng 2.1: Bảng tín hiệu chấm điểm cho candidate tên người

Tín hiệu	Điểm	Ghi chú
Trigger mạnh + candidate đứng ngay sau	100, break	Dừng ngay, không so sánh
I, [name] hoặc [name], I	100, break	
I'm [name] hoặc I am [name]	100, break	
I was named [name]	100, break	
Từ my name xuất hiện trong câu	+10	
As [name], đứng đầu câu	+5	
Dấu : đứng ngay trước candidate	+5	Bất kỳ label nào
Appositive: [name], <chức danh> ,	+3	Từ sau phải là nghề nghiệp
Chữ ký: [name] \n<chức danh>	+4	Candidate trên dòng riêng
Đại từ ngôi 1 (I, me, my) trong câu	+2	Ưu tiên hơn ngôi 3
Đại từ ngôi 3 (he, she, his, her)	+1	Chỉ tính khi không có ngôi 1
Nghề nghiệp xuất hiện trong cùng câu	+2	Xem phần phát hiện nghề nghiệp
Tên lặp lại trong toàn đoạn (partial match)	+1/lần	Tối đa +5
Title prefix (Dr., Mr., v.v.) trước tên	+1	

Lưu ý rằng đại từ không cộng dồn nếu xuất hiện nhiều lần trong cùng một câu. Nếu câu đồng thời có đại từ ngôi 1 và ngôi 3 thì chỉ tính điểm cho ngôi 1.

Phát hiện nghề nghiệp. Hàm `_has_job_in_sentence(sentence)` trả về True nếu câu chứa tín hiệu nghề nghiệp theo một trong hai cách: (1) từ nằm trong danh sách cứng khoảng 35 nghề (doctor, lawyer, developer, CEO, v.v.) bất kể vị trí trong câu; (2) từ mang đuôi nghề nghiệp (-er, -ist, -or, -ician, v.v.) và đứng ngay sau một role marker như a, an, is a, works as. Điều kiện role marker giúp tránh false positive như underwater hay after.



Bước 2.3 — Chọn winner. Sau khi chấm điểm toàn bộ candidate, winner được chọn theo quy tắc: nếu có candidate đạt điểm 100 (definitive) thì trả về ngay; nếu chỉ có một candidate thì trả về candidate đó dù điểm bằng 0; nếu nhiều candidate thì chọn điểm cao nhất, tie-breaking theo vị trí xuất hiện sớm hơn trong văn bản.

Algorithm 1 Thuật toán nhận diện tên người

Require: Văn bản đầu vào T

Ensure: Danh sách thực thể tên E

```
1:  $E \leftarrow \emptyset$ 
2:  $lower \leftarrow \text{lowercase}(T)$ 
// Tầng 1: Fast path
3: for mỗi trigger  $tr$  trong  $\text{NAMETRIGGERS} \cup \text{LABELTRIGGERS}$  do
4:   for mỗi vị trí  $pos$  của  $tr$  trong  $lower$  do
5:     Thu thập 1–4 token hoa liên tiếp sau  $pos$  vào  $tokens$ 
6:     if  $tokens \neq \emptyset$  then
7:       Thêm  $\text{join}(tokens)$  vào  $E$  với điểm 100
8:     end if
9:   end for
10: end for
11: if  $E \neq \emptyset$  then return  $E$ 
12: end if
// Tầng 2: Scoring path
13:  $sentences \leftarrow \text{SPLITSENTENCES}(T)$ 
14:  $candidates \leftarrow \text{EXTRACTCANDIDATES}(T)$ 
15: if  $candidates = \emptyset$  then return  $\emptyset$ 
16: end if
17:  $bestScores \leftarrow \{\}$  //  $candidate \rightarrow (score, start, end)$ 
18: for mỗi candidate  $c$  trong  $candidates$  do
19:    $best \leftarrow 0$ 
20:   for mỗi câu  $s$  trong  $sentences$  do
21:     if không có phần nào của  $c$  trong  $s$  then continue
22:   end if
23:    $(score, def) \leftarrow \text{SCORECANDIDATE}(c, s, T)$ 
24:   if  $def = \text{True}$  then
25:     return  $\{c\}$  // Definitive, dừng ngay
26:   end if
27:    $best \leftarrow \max(best, score)$ 
28: end for
29: Cập nhật  $bestScores[c]$  nếu  $best$  cao hơn giá trị hiện tại
30: end for
31:  $winner \leftarrow \arg \max_c bestScores[c]$  // Tie: chọn cái xuất hiện sớm hơn
32: return  $\{winner\}$ 
```

2.2.2 Nhận diện địa chỉ

Địa chỉ trong tập dữ liệu tuân theo cấu trúc điển hình của địa chỉ Mỹ gồm số nhà, tên đường, loại đường, và hậu tố hướng hoặc đơn vị căn hộ tùy chọn. Module nhận diện địa chỉ sử dụng hai cơ chế bổ sung cho nhau.

Cơ chế 1 — Regex pattern. Biểu thức chính quy `_ADDRESS_RE` mô tả cấu trúc địa chỉ theo bốn thành phần nối tiếp:

1. **Số nhà:** `\b(\d{1,5})(?!\d)\s+` — từ 1 đến 5 chữ số, kèm negative lookahead `(?!\d)` để đảm bảo đây không phải là một phần của số dài hơn (loại trừ số điện thoại, mã bưu điện dài).
2. **Tên đường:** `((?:[A-Z0-9][A-Za-z0-9]*\s+)+?)` — một hoặc nhiều từ bắt đầu bằng chữ hoa hoặc chữ số, sử dụng non-greedy để nhường ưu tiên cho thành phần loại đường phía sau.
3. **Loại đường:** danh sách khoảng 30 từ được sắp xếp từ dài đến ngắn để tránh match sớm, bao gồm dạng đầy đủ (Boulevard, Terrace, Avenue, Street, Drive, Road, v.v.) và dạng viết tắt (Blvd., Ave., Rd., St., v.v.).
4. **Hướng và đơn vị** (tùy chọn): hướng gồm tám giá trị North, South, East, West và các tổ hợp; đơn vị gồm Suite, Apt., Apartment, Unit kèm số phòng.

Trước khi kiểm tra kết quả regex, tất cả span email trong văn bản được xác định trước; bất kỳ span địa chỉ nào nằm hoàn toàn bên trong span email sẽ bị bỏ qua để tránh nhầm lẫn phần tên miền của địa chỉ email với địa chỉ nhà.

Cơ chế 2 — Trigger keyword (fallback). Đối với địa chỉ không có loại đường tiêu chuẩn, module sử dụng danh sách trigger bao gồm `street`, `road`, `avenue`, `district`, `city`, `st.`, `rd.`, `ave.`. Khi tìm thấy trigger, thuật toán lấy clause xung quanh (từ dấu câu trước đến dấu câu sau), loại bỏ noise words đầu clause (`at`, `is`, `the`, `located`, `live`, `reside`, v.v.), và kiểm tra thêm: số nhà ở đầu clause không được vượt quá 5 chữ số, và span không được trùng lặp với kết quả từ cơ chế regex.

Algorithm 2 Thuật toán nhận diện địa chỉ

Require: Văn bản đầu vào T

Ensure: Danh sách thực thể địa chỉ E

```
1:  $E \leftarrow \emptyset$ 
2:  $emailSpans \leftarrow \text{FINDEMAILSPANS}(T)$ 
// Cơ chế 1: Regex pattern
3: for mỗi match  $m$  của ADDRESSRE trong  $T$  do
4:   if  $m$  nằm trong một email span then continue
5:   end if
6:   Thêm span của  $m$  vào  $E$ 
7: end for
// Cơ chế 2: Trigger keyword fallback
8: for mỗi trigger  $tr$  trong ADDRESSTRIGGERS do
9:   for mỗi vị trí  $pos$  của  $tr$  trong  $T$  do
10:     $clause \leftarrow \text{EXTRACTCLAUSE}(T, pos)$ 
11:     $addr \leftarrow \text{STRIPLEADINGNOISE}(clause)$ 
12:    if số nhà đầu  $addr$  có nhiều hơn 5 chữ số then continue
13:    end if
14:    if  $addr$  nằm trong email span then continue
15:    end if
16:    if  $addr$  overlap với span đã có trong  $E$  then continue
17:    end if
18:    Thêm span của  $addr$  vào  $E$ 
19:   end for
20: end for
21: return  $\text{DEDUPLICATE}(E)$ 
```

2.3 Hợp nhất thực thể (Merger)

Sau khi `regex_detector` và `context_detector` chạy độc lập, mỗi module trả về một danh sách thực thể riêng. Hai danh sách này được đưa vào module `merger` để hợp nhất thành một danh sách duy nhất, sạch và không chồng lấn.

2.3.1 Vấn đề overlap

Hai thực thể được coi là *chồng lấn* (overlap) nếu khoảng ký tự của chúng giao nhau, tức là $[s_1, e_1)$ và $[s_2, e_2)$ thỏa $s_1 < e_2$ và $e_1 > s_2$. Overlap xảy ra trong nhiều tình huống thực tế:

- Địa chỉ email `kazuosun@hotmail.net` bị `regex_detector` bắt đúng là **EMAIL**, nhưng `context_detector` có thể nhận nhầm phần `hotmail.net` là URL hoặc nhận phần trước `@` là **NAME**.
- Số điện thoại `+27 68 670 7513` có thể bị nhận vừa là **PHONE** vừa overlap với một span **ADDRESS** nếu đứng sau một địa chỉ.
- Tên người `Baha Hoffman` xuất hiện ngay trước số điện thoại, `context_detector` có thể kéo dài span sang phần số.

2.3.2 Hệ thống ưu tiên

Để giải quyết overlap một cách nhất quán, merger định nghĩa một thứ tự ưu tiên tuyến tính cho sáu loại thực thể:

EMAIL > URL > PHONE > ADDRESS > NAME > USERNAME

Thứ tự này phản ánh độ tin cậy của từng detector: **EMAIL**, **URL** và **PHONE** được phát hiện bằng regex có độ chính xác cao và cấu trúc rõ ràng nên được ưu tiên hơn; **ADDRESS**, **NAME**, **USERNAME** dựa trên ngữ cảnh nên được xếp sau.

2.3.3 Thuật toán Greedy Priority-First

Hàm `resolve_overlaps` thực hiện giải quyết overlap theo chiến lược tham lam ưu tiên (greedy priority-first):

1. Gộp tất cả thực thể từ mọi detector thành một danh sách duy nhất.
2. Sắp xếp danh sách theo ba tiêu chí lần lượt: (a) thứ hạng ưu tiên tăng dần (loại ưu tiên cao xếp trước), (b) chỉ số **start** tăng dần, (c) độ dài span giảm dần (span dài hơn xếp trước khi cùng điểm xuất phát).
3. Duyệt tuần tự qua danh sách đã sắp xếp. Với mỗi thực thể, giữ lại nếu span của nó không chồng lấn với bất kỳ thực thể đã giữ nào; bỏ qua nếu ngược lại.
4. Sắp xếp lại danh sách kết quả theo **start** để đảm bảo thứ tự xuất hiện trong văn bản.

Algorithm 3 Thuật toán hợp nhất và giải quyết overlap (Greedy Priority-First)

Require: Các nhóm thực thể $G_1, G_2, \dots, G_k$ từ các detector

Ensure: Danh sách thực thể không overlap, sắp theo vị trí

```
1:  $all \leftarrow G_1 \cup G_2 \cup \dots \cup G_k$ 
2: Sắp xếp  $all$  theo ( $priority(type)$ ,  $start$ ,  $-(end - start)$ ) tăng dần
3:  $kept \leftarrow \emptyset$ 
4:  $occupied \leftarrow \emptyset$  // Tập các  $span(s, e)$  đã giữ
5: for mỗi thực thể  $ent$  trong  $all$  do
6:    $s \leftarrow ent.start$ ,  $e \leftarrow ent.end$ 
7:   if  $\nexists (os, oe) \in occupied$  sao cho  $s < oe$  và  $e > os$  then
8:      $kept \leftarrow kept \cup \{ent\}$ 
9:      $occupied \leftarrow occupied \cup \{(s, e)\}$ 
10:  end if
11: end for
12: Sắp xếp  $kept$  theo  $start$  tăng dần
13: return  $kept$ 
```

2.3.4 Minh họa trường hợp overlap

Xét đoạn văn bản sau được trích từ tập dữ liệu:

*“please do not hesitate to contact Baha Hoffman at +27 68 670 7513 or via email at
bahahoffman@yahoo.net.”*

Hai detector tạo ra các thực thể như trong Bảng 2.2.

Bảng 2.2: Danh sách thực thể trước khi hợp nhất

Detector	Loại	Text	Ưu tiên
regex_detector	PHONE	+27 68 670 7513	3
regex_detector	EMAIL	bahahoffman@yahoo.net	1
context_detector	NAME	Baha Hoffman	5
context_detector	NAME	Baha Hoffman at	5

Sau khi sắp xếp theo ưu tiên và áp dụng thuật toán greedy, merger loại bỏ Baha

Hoffman at vì overlap với Baha Hoffman đã được giữ trước đó (cùng loại NAME nhưng span ngắn hơn xếp trước). Kết quả cuối cùng được trình bày trong Bảng 2.3.

Bảng 2.3: Danh sách thực thể sau khi hợp nhất

Loại	Text	Vị trí
NAME	Baha Hoffman	start=35, end=47
PHONE	+27 68 670 7513	start=51, end=66
EMAIL	bahahoffman@yahoo.net	start=79, end=103

Kết quả này sau đó được đưa vào module `anonymizer` để thay thế các span PII bằng placeholder tương ứng, tạo ra văn bản đã được ẩn danh hóa.

3 Tổng kết

Trong project này, nhóm đã xây dựng một hệ thống nhận diện và ẩn danh thông tin cá nhân trong văn bản bằng phương pháp rule-based. Hệ thống tập trung vào việc phát hiện các thực thể quan trọng như NAME, EMAIL, PHONE, ADDRESS, USERNAME và URL. Thay vì sử dụng các mô hình machine learning hoặc deep learning, project áp dụng các kỹ thuật xử lý chuỗi như regular expression, keyword matching, token scanning và xử lý overlap giữa các entity để giải quyết bài toán NER.

Trong quá trình thực hiện, nhóm đã xây dựng được một pipeline xử lý tương đối hoàn chỉnh, bao gồm các bước preprocessing dữ liệu, phát hiện entity bằng regex, phát hiện entity bằng ngữ cảnh, gộp kết quả, xử lý entity chồng lấn, ẩn danh văn bản và đánh giá kết quả bằng các chỉ số Precision, Recall và F1-score. Việc chia hệ thống thành nhiều module độc lập giúp các thành viên có thể phát triển song song, đồng thời hỗ trợ quá trình kiểm thử và mở rộng sau này trở nên dễ dàng hơn.

Kết quả đạt được cho thấy phương pháp rule-based hoạt động tương đối hiệu quả đối với các thực thể có cấu trúc rõ ràng như email, URL và số điện thoại. Các loại thực thể này thường có format tương đối cố định nên phù hợp với phương pháp regular expression. Đối với các thực thể phụ thuộc nhiều vào ngữ cảnh như tên người hoặc địa chỉ, hệ thống vẫn có thể phát hiện được bằng cách sử dụng các luật thủ công dựa trên keyword và context. Ngoài ra, cơ chế xử lý overlap giữa các entity cũng giúp giảm một phần lỗi nhận diện trùng lặp hoặc phát hiện sai trong văn bản.

Bên cạnh những kết quả đạt được, project vẫn còn tồn tại một số hạn chế nhất định. Vì sử dụng rule-based method nên chất lượng nhận diện phụ thuộc rất nhiều vào tập luật được xây dựng thủ công. Nếu văn bản có cấu trúc quá đa dạng hoặc xuất hiện các format hiếm gặp, hệ thống có thể bỏ sót entity hoặc nhận diện sai. Đặc biệt, các thực thể như tên người và địa chỉ thường có rất nhiều biến thể khác nhau nên khó đạt độ chính xác cao chỉ bằng các luật đơn giản. Ngoài ra, phương pháp đánh giá exact match trong bài toán NER cũng khá nghiêm ngặt, chỉ cần lệch một phần nhỏ vị trí span thì prediction sẽ bị xem là sai hoàn toàn.

Mặc dù vậy, project vẫn thể hiện rõ vai trò quan trọng của bài toán NER trong thời đại AI hiện nay. Khi lượng dữ liệu văn bản ngày càng lớn và được sử dụng trong nhiều hệ thống khác nhau, nhu cầu bảo vệ thông tin cá nhân ngày càng trở nên cần thiết. Các kỹ thuật NER có thể hỗ trợ tự động phát hiện và ẩn danh dữ liệu nhạy cảm trước khi lưu trữ, chia sẻ hoặc xử lý tiếp. Điều này đặc biệt quan trọng trong các lĩnh vực như tài chính, y tế, giáo dục, thương mại điện tử và mạng xã hội, nơi dữ liệu cá nhân thường

xuyên được xử lý với số lượng lớn.

Thông qua project này, nhóm không chỉ hiểu rõ hơn về bài toán nhận diện thực thể trong xử lý ngôn ngữ tự nhiên mà còn có cơ hội áp dụng các thuật toán xử lý chuỗi, regular expression và thiết kế pipeline xử lý dữ liệu thực tế. Đây cũng là nền tảng quan trọng để tiếp cận các hướng nghiên cứu hoặc hệ thống NER phức tạp hơn trong tương lai.

4 Hướng mở rộng trong tương lai

Trong tương lai, hệ thống có thể được mở rộng theo nhiều hướng khác nhau nhằm cải thiện độ chính xác, khả năng tổng quát hóa và khả năng ứng dụng thực tế của bài toán NER. Một trong những hướng phát triển quan trọng là bổ sung thêm rule và dictionary để tăng khả năng nhận diện tên riêng, địa danh và địa chỉ. Việc sử dụng danh sách first name, last name, tên đường hoặc địa danh phổ biến có thể giúp giảm false positive và tăng độ chính xác cho các context-based detector.

Ngoài ra, project hiện chủ yếu hoạt động trên dữ liệu tiếng Anh nên vẫn chưa tối ưu cho tiếng Việt. Trong tương lai, hệ thống có thể được mở rộng sang dữ liệu tiếng Việt bằng cách xây dựng thêm các rule phù hợp với cấu trúc tên người, số điện thoại và địa chỉ tại Việt Nam. Đây là hướng mở rộng có ý nghĩa thực tế cao vì nhiều hệ thống trong nước hiện vẫn gặp khó khăn trong việc xử lý dữ liệu tiếng Việt.

Một hướng cải tiến khác là tối ưu quá trình xử lý overlap giữa các entity. Hiện tại, project chủ yếu sử dụng cơ chế priority-based greedy selection để xử lý entity chồng lấn. Trong tương lai, có thể áp dụng các kỹ thuật như interval scheduling hoặc weighted interval optimization để lựa chọn tập entity tối ưu hơn. Ngoài ra, fuzzy matching cũng có thể được tích hợp nhằm xử lý các trường hợp entity bị viết tắt hoặc có sai lệch nhỏ về format.

Bên cạnh đó, hệ thống có thể được mở rộng thêm nhiều loại entity mới ngoài phạm vi hiện tại, chẳng hạn như tên tổ chức, mã số định danh, biển số xe, tài khoản ngân hàng hoặc thông tin học tập. Việc mở rộng tập entity sẽ giúp hệ thống phù hợp hơn với các bài toán anonymization thực tế trong doanh nghiệp hoặc các hệ thống lưu trữ dữ liệu lớn.

Một hướng phát triển quan trọng khác là kết hợp rule-based method với machine learning hoặc deep learning nhằm tận dụng ưu điểm của cả hai phương pháp. Rule-based approach có tính giải thích cao và hoạt động tốt với các mẫu cố định, trong khi machine learning có khả năng học được các pattern phức tạp và tổng quát hóa tốt hơn trên dữ liệu đa dạng. Việc kết hợp hybrid approach có thể giúp cải thiện đáng kể hiệu năng của

hệ thống NER trong các ngữ cảnh thực tế.

Ngoài phần xử lý backend, project cũng có thể được mở rộng bằng cách xây dựng giao diện web local để trực quan hóa kết quả nhận diện entity. Người dùng có thể nhập văn bản trực tiếp lên website, sau đó hệ thống sẽ highlight các thực thể được phát hiện và hiển thị phiên bản đã được ẩn danh. Điều này giúp project dễ demo hơn và tăng tính ứng dụng thực tế của hệ thống.

Tóm lại, dù vẫn còn nhiều hạn chế, project đã xây dựng được một nền tảng tương đối hoàn chỉnh cho bài toán nhận diện và ẩn danh thông tin cá nhân bằng phương pháp rule-based. Các hướng mở rộng trong tương lai không chỉ giúp cải thiện độ chính xác của hệ thống mà còn mở ra khả năng ứng dụng rộng hơn trong các bài toán NLP và bảo vệ dữ liệu cá nhân trong thời đại AI.